

Lab 08 – Doubly Linked List

Task 01:

Design a user-friendly navigation application to assist commuters traveling between their home and Bahria University. The application must utilize a doubly linked list data structure to store the sequence of directions for the route. Users should be able to add new route steps, delete the entire route, and seamlessly navigate between home and university. Clear and concise navigation instructions, including turning directions and approximate distances, must be provided for each route step. Also show the reverse the route to navigate from Bahria University back to the user's home.

Code:

```
#include<iostream>
using namespace std;

class Node{
public:
    string step;
    Node* pre;
    Node* next;

    Node(string step){
        this->step = step;
        pre = next = nullptr;
    }
};

class UniversityRoute{
private:
    Node* head;
    Node* tail;

public:
    UniversityRoute(){ head = tail = nullptr; }

    // 1. add new route step
    void addRouteStep(string val){
        if(head == nullptr){
            head = tail = new Node(val);
        } else {
            Node* newNode = new Node(val);

            newNode->pre = tail;
            tail->next = newNode;
        }
    }
};
```

```
        tail = newNode;
    }
}

// 2. delete entire route
void deleteEntireRoute(){
    if(head == nullptr){
        cout << "No route step to delete\n";
    } else {
        while(head != nullptr){
            Node* temp = head;

            head = head->next;
            delete temp;
        }
        head = tail = nullptr;
        cout << "All steps deleted.\n";
    }
}

// 3. show route in reverse order
void displayReverse(){
    if(head == nullptr){
        cout << "No route to display.\n";
    } else {
        Node* temp = tail;

        cout << "Route Steps:\n\n";
        int count = 1;
        while(temp != nullptr){
            cout << count++ << ". " << temp->step << "\n";
            temp = temp->pre;
        }
    }
}

};

int main(){
    UniversityRoute route;

    route.addRouteStep("Start from Home");
    route.addRouteStep("Go straight for 500m");
    route.addRouteStep("Turn left at the traffic signal");
    route.addRouteStep("Continue for 1.2 km");
    route.addRouteStep("Turn right towards Bahria University");
```

```
route.addRouteStep("You have reached Bahria University");

cout<<"=====  
Bahria University ---> Home =====\n";
route.displayReverse();

cout<<"\nDeleting Entire Route...\n";
route.deleteEntireRoute();

cout<<"\nTrying to display after deletion:\n";
route.displayReverse();
}
```

Output:

```
=====  
Bahria University ---> Home =====  
Route Steps:  
  
1. You have reached Bahria University  
2. Turn right towards Bahria University  
3. Continue for 1.2 km  
4. Turn left at the traffic signal  
5. Go straight for 500m  
6. Start from Home  
  
Deleting Entire Route...  
All steps deleted.  
  
Trying to display after deletion:  
No route to display.
```

Task 02:

Alex is an avid card game player and enjoys playing different strategy-based card games during his free time. Over the years, Alex has developed his own deck of cards, which he frequently updates by adding or removing cards to create new strategies for his games. Now, he wants to manage his custom deck more efficiently. Each card in his deck is characterized by its card rank (e.g., Ace, 2, 3, ..., King) and card suit (e.g., Hearts, Diamonds, Clubs, Spades).

Your task is to develop a C++ program that helps Alex manage his deck of cards using a **doubly linked list**. The program should incorporate the following functionalities:

1. **Adding New Cards:** Alex should be able to add new cards to his deck. Each new card will be appended to the **top** of the deck.
2. **Displaying the Deck:** The program should enable Alex to view his deck in both **forward** and **backward** directions, allowing him to see the order of the cards from the top to the bottom or in reverse order.
3. **Inserting Cards After a Specific Card:** Alex should be able to insert a new card after a specific card in the deck. He will specify the card (by rank and suit) after which the new card will be inserted.
4. **Searching by Rank and Suit:** The program should provide functionality for Alex to search for a card in the deck based on its rank and suit. Upon searching, the program should display the position of the card in the deck.
5. **Removing from the Top:** Alex should be able to remove the top card from his deck, simulating a card draw.

Code:

```
#include<iostream>
using namespace std;

struct Card{
    string rank;
    string suit;
};

class Node{
public:
    Card data;
    Node* pre;
    Node* next;

    Node(Card data){
        this->data = data;
        pre = next = nullptr;
    }
}
```

```
};

class CardDeck{
private:
    Node *head, *tail;

public:
    CardDeck(){ head = tail = nullptr; }

    // 1. Adding new card (at top)
    void addNewCard(Card val){
        if(head == nullptr){
            head = tail = new Node(val);
        } else {
            Node* newNode = new Node(val);

            newNode->next = head;
            head->pre = newNode;
            head = newNode;
        }
    }

    // 2. Display the card (in forward order)
    void displayCards(){
        if(head == nullptr){
            cout << "No cards to display.\n";
        } else {
            Node* temp = head;

            cout << "\t---Cards in the deck---\n";
            while(temp){
                cout << "Card rank: " << temp->data.rank << ", Card suit: " <<
temp->data.suit << "\n";
                temp = temp->next;
            }
            cout << "\n";
        }
    }

    // 3. Inserting Cards After a Specific Card
    void insertByRank(Card val, string target){
        if(head == nullptr){
            head = tail = new Node(val);
            return;
        }
    }
}
```

```
// simple linear search
Node* temp = head;

while(temp != nullptr){
    if(temp->data.rank == target) break;
    temp = temp->next;
}

if(!temp){
    cout << "Card with this rank doesn't exists.\n";
    return;
}

Node* newNode = new Node(val);
newNode->pre = temp;
newNode->next = temp->next;

if(temp == tail){
    temp->next = newNode;
    tail = newNode;
}
else{
    temp->next->pre = newNode;
    temp->next = newNode;
}
}

// 4. Search by rank (if present then display with position)
void searchByrank(string target){
    if(head == nullptr){
        cout << "There are no cards in deck to search from.\n";
        return;
    }

    Node* temp = head;
    int count = 1;
    while(temp){
        if(temp->data.rank == target) break;
        temp = temp->next;
        count++;
    }

    if(!temp){
        cout << "No card with the this rank exists in the deck.\n";
    }
}
```

```
        return;
    }

    cout << "Card details:\n";
    cout << "\tPosition: " << count << "\n";
    cout << "\tRank: " << temp->data.rank << "\n";
    cout << "\tSuit: " << temp->data.suit << "\n";
}

// 5. Remove form top
void removeFromTop(){
    if(head == nullptr){
        cout << "No card to remnove from top.\n";
        return;
    }

    Node* temp = head;
    head = head->next;

    if(head == nullptr) tail = nullptr;
    else head->pre = nullptr;

    delete temp;
}

};

int main(){
    CardDeck deck;

    Card c1 = {"Ace", "Spades"};
    Card c2 = {"King", "Hearts"};
    Card c3 = {"Queen", "Diamonds"};
    Card c4 = {"Jack", "Clubs"};
    Card c5 = {"10", "Spades"};

    deck.addNewCard(c1);
    deck.addNewCard(c2);
    deck.addNewCard(c3);

    cout<<"Initial Deck\n";
    deck.displayCards();

    cout<<"\nSearching Queen\n";
    deck.searchByrank("Queen");
```

```
    cout<<"\nInserting after King\n";
    deck.insertByRank(c4,"King");

    deck.displayCards();

    cout<<"\nRemoving top card\n";
    deck.removeFromTop();

    deck.displayCards();
}
```

Output:

```
● Initial Deck
    ---Cards in the deck---
Card rank: Queen, Card suit: Diamonds
Card rank: King, Card suit: Hearts
Card rank: Ace, Card suit: Spades

Searching Queen
Card details:
    Position: 1
    Rank: Queen
    Suit: Diamonds

Inserting after King
    ---Cards in the deck---
Card rank: Queen, Card suit: Diamonds
Card rank: King, Card suit: Hearts
Card rank: Jack, Card suit: Clubs
Card rank: Ace, Card suit: Spades

Removing top card
    ---Cards in the deck---
Card rank: King, Card suit: Hearts
Card rank: Jack, Card suit: Clubs
Card rank: Ace, Card suit: Spades
```


Task 03:

Sara is passionate about music and enjoys listening to music during her free time. Over the years, she has amassed a vast collection of music albums comprising her favorite songs. Now, she desires to organize her favorite songs into a new playlist named "My Hit Collection". Each song in her playlist is characterized by its Song ID, song name, singer's name, and the year the song was released.

Your task is to develop a C++ program that helps Sara manage her music playlist using a doubly linked list. The program should incorporate the following functionalities:

1. **Adding New Songs:** Sara should be able to add new songs to her playlist. Each new song will be appended to the end of the playlist.
2. **Playing Playlist:** The program should enable Sara to play her playlist in both forward and backward directions, allowing her to enjoy her favorite songs in the sequence they were added or in reverse order.
3. **Circular Playback:** Sara should have the option to play her playlist in a circular fashion, ensuring a seamless listening experience without interruption.
4. **Searching by Year:** The program should provide functionality for Sara to search for songs based on the year of release. Upon searching, the program should display all songs released in the specified year.
5. **Deleting from the Beginning:** Sara should be able to remove songs from the beginning of the playlist, enabling her to manage her playlist dynamically.

Your program should efficiently implement these functionalities to cater to Sara's needs and enhance her music listening experience.

Code:

```
#include<iostream>
#include <string>
using namespace std;

struct Song{
    int id;
    string name;
    string singer;
    int year;
};

class Node{
public:
    Song data;
    Node* pre;
    Node* next;

    Node(Song data){
```

```
        this->data = data;
        pre = next = nullptr;
    }
};

class MyHitPlaylist{
private:
    Node *head, *tail;

public:
    MyHitPlaylist(){ head = tail = nullptr; }

    // 1. Add new song. Each song will append to end
    void addSong(Song val){
        if(head == nullptr){
            head = tail = new Node(val);
            head->next = head;
            head->pre = head;
            return;
        }

        Node* newNode = new Node(val);

        newNode->pre = tail;
        tail->next = newNode;
        tail = newNode;

        tail->next = head;
        head->pre = tail;
    }

    // 2. Play in backward order
    void playBackward(){
        if(head == nullptr){
            cout << "No songs in playlist to play in backward order.\n";
            return;
        }

        Node* temp = tail;

        cout << "\t---Songs in the playlist---\n";
        do{
            cout << "ID: " << temp->data.id << " | Name: " << temp->data.name <<
            " | Singer: " << temp->data.singer
```

```

        << " | Released year: " << temp->data.year << "\n";
        temp = temp->pre;

    }while(temp != tail);

}

// 3. Circular play back
void circularPlayBack(){
    if(head == nullptr){
        cout << "No songs in playlist to play.\n";
        return;
    }

    cout << "\nIMPORTANT: To stop press: Ctrl + c\n";
    string startPlaying;
    cout << "Press enter to continue... ";
    cin >> startPlaying;

    Node* temp = head;
    while(temp){
        cout << "Playing... " << temp->data.name << "Singer: " << temp->data.singer << "\n";
        temp = temp->next;
    }
}

// 4. Search by year
void searchByYear(int target){
    if(head == nullptr){
        cout << "No songs in playlist.\n";
        return;
    }
    cout << "Songs released in " << target << "\n";

    Node* temp = head;
    int count = 1;
    do{
        if(temp->data.year == target){
            cout << count++ << ". Name: " << temp->data.name << ", Singer: "
<< temp->data.singer << "\n";
        }
        temp = temp->next;
    }while(temp != head);
}

```

```
// 5. delete from begining
void deleteFromBegin(){
    if(head == nullptr){
        cout << "No song to delete.\n";
        return;
    }

    Node* temp = head;
    head = head->next;

    if(!head) head = tail = nullptr;
    else{ tail->next = head; head->pre = nullptr; }

    delete temp;
}
};

int main(){

    MyHitPlaylist playlist;

    Song s1 = {1,"Wake me up","aleemrk",2024};
    Song s2 = {2,"Uljhan","Nabeel Akbar",2022};
    Song s3 = {3,"Kaha tak","Ka$na",2023};
    Song s4 = {4,"Locked Doors","JJ47",2025};
    Song s5 = {5,"Qalam","AHSAN",2022};

    playlist.addSong(s1);
    playlist.addSong(s2);
    playlist.addSong(s3);
    playlist.addSong(s4);
    playlist.addSong(s5);

    cout<<"Backward Playlist\n";
    playlist.playBackward();

    cout<<"\nSearching songs released in 2017\n";
    playlist.searchByYear(2017);

    cout<<"\nDeleting first song\n";
    playlist.deleteFromBegin();

    cout<<"\nBackward Playlist After Deletion\n";
    playlist.playBackward();
```

```
    playlist.circularPlayBack();  
}
```

Output:

```
Backward Playlist  
    ---Songs in the playlist---  
ID: 5 | Name: Qalam | Singer: AHSAN | Released year: 2022  
ID: 4 | Name: Locked Doors | Singer: JJ47 | Released year: 2025  
ID: 3 | Name: Kaha tak | Singer: Ka$na | Released year: 2023  
ID: 2 | Name: Uljhan | Singer: Nabeel Akbar | Released year: 2022  
ID: 1 | Name: Wake me up | Singer: aleemrk | Released year: 2024  
  
Searching songs released in 2017  
Songs released in 2017  
  
Deleting first song  
  
Backward Playlist After Deletion  
    ---Songs in the playlist---  
ID: 5 | Name: Qalam | Singer: AHSAN | Released year: 2022  
ID: 4 | Name: Locked Doors | Singer: JJ47 | Released year: 2025  
ID: 3 | Name: Kaha tak | Singer: Ka$na | Released year: 2023  
ID: 2 | Name: Uljhan | Singer: Nabeel Akbar | Released year: 2022
```